

Title

Autonomous Intermittent Failure Scientist: An Uncertainty-Guided Framework for Discovery and Minimization of Multi-Factor Software Failure Triggers

Background

Modern software systems increasingly consist of distributed services, databases, external APIs, asynchronous workers, retry mechanisms, and concurrent processes. Failures in such systems are often not caused by a single defect or input. Instead, they emerge only under rare combinations of conditions such as network latency, concurrent requests, retry timing, database state, resource pressure, service failures, and execution order. Because these failures occur inconsistently and are difficult to reproduce, developers may spend substantial time manually modifying system conditions and repeatedly executing tests to identify the circumstances responsible for the failure.

Problem

Intermittent software failures may occur only when multiple environmental and execution conditions interact within specific ranges or temporal sequences. A system may function correctly under high latency alone, concurrency alone, or retries alone, but fail when these conditions occur together. Developers typically receive only an observed symptom, such as duplicate transactions, inconsistent database state, request timeout, or partial operation completion, without knowing the exact trigger conditions.

The central problem addressed by this project is:

How can a software system autonomously discover and minimize the combination of conditions responsible for an intermittent failure without exhaustively testing the entire configuration space?

Existing Limitations

Current approaches have several limitations. Manual debugging depends heavily on developer experience and repeated trial-and-error experimentation. Random testing and fuzzing may generate useful failures but can waste large numbers of executions exploring irrelevant regions. Combinatorial testing becomes computationally expensive as the number of parameters and parameter values increases. Fault-injection systems can introduce failures but generally require engineers to define the experiments in advance. Delta debugging can minimize an already reproducible failure-inducing input or configuration but does not by itself solve the broader problem of efficiently discovering an unknown multi-factor trigger.

Consequently, existing methods often fail to answer three questions efficiently:

- Which experiment should be executed next?
- Which combination of conditions is actually responsible for the failure?
- What is the smallest reliable trigger that can reproduce the failure?

Research Gap

Most automated software testing techniques focus on generating inputs, injecting predefined faults, exploring configurations, or minimizing already discovered failures. There is a need for a closed-loop diagnostic framework that treats intermittent failure reproduction as a sequential experimental discovery problem.

The research gap investigated by this project is whether an autonomous system can continuously:

observe previous experiments → estimate uncertainty → select the most informative next experiment → discover a failure region → minimize its trigger conditions → validate reproducibility

The project specifically investigates whether uncertainty-guided sequential experimentation can discover rare multi-factor software failure triggers using fewer system executions than non-adaptive diagnostic search methods.

Proposed Solution

The proposed system, called the **Autonomous Intermittent Failure Scientist**, treats software debugging as an autonomous scientific experimentation process.

The system receives:

1. a software system under test;
2. a description or observable definition of the failure;
3. a configurable set of environmental and execution variables.

It then autonomously performs the following cycle:

Generate candidate experiment → Select parameter configuration → Inject controlled conditions → Execute system → Observe behaviour → Determine failure outcome → Update experimental model → Select next experiment

The system progressively learns which regions of the configuration space are most likely to produce the target failure. Once a failure is discovered, it switches from exploration to trigger minimization and attempts to identify the smallest set and narrowest ranges of conditions required for reliable reproduction.

The final output is an evidence-backed failure report containing the discovered trigger conditions, reproduction probability, experimental history, minimized failure configuration, and an automatically generated regression test.

Novel Contribution

The primary proposed contribution is an **uncertainty-guided autonomous experimentation framework for discovering and minimizing multi-factor intermittent software failure triggers**.

Rather than executing experiments according to a fixed schedule, random search, or exhaustive enumeration, the system uses evidence from previous executions to decide which experiment should be performed next.

The proposed novelty consists of three connected mechanisms:

1. **Adaptive experiment selection** — selecting the next system configuration according to uncertainty and expected information value.
2. **Failure-boundary discovery** — identifying regions where small changes in latency, concurrency, timing, state, or resources cause transitions between successful and failed executions.
3. **Reliability-aware trigger minimization** — reducing a discovered failure configuration to the smallest combination of conditions that still reproduces the failure with a measurable probability.

The research contribution is therefore not merely automated fault injection or AI-generated debugging advice. It is the closed-loop integration of autonomous experimental planning, probabilistic failure discovery, and trigger minimization.

Methodology

The software system is represented as an experimental environment containing controllable variables such as:

$X = \{\text{latency, concurrency, retry timing, packet loss, CPU pressure, memory pressure, database state, service availability, execution order}\}$

Each experiment selects a configuration:

$$x_i \in X$$

The system executes the software under that configuration and observes an outcome:

$y_i = \text{failure or success}$

After each experiment, a surrogate model estimates:

$P(\text{Failure} \mid x)$

and the uncertainty associated with that prediction.

An acquisition strategy then selects the next experiment based on criteria such as:

- predictive uncertainty;
- expected information gain;
- probability of failure;
- proximity to an estimated failure boundary;
- novelty relative to previous experiments.

The project will investigate suitable techniques such as Bayesian optimization, active learning, probabilistic surrogate modelling, and uncertainty-based acquisition functions.

After discovering a failure-producing region, the system performs trigger minimization by removing conditions, reducing parameter ranges, and repeatedly validating candidate configurations. The final trigger is accepted only after repeated executions establish its reproduction reliability.

Core Functionalities / Modules

Module 1: Experimental Environment and Fault-Injection Sandbox

Creates a controlled environment in which system conditions can be manipulated safely and repeatedly.

It controls network latency, packet loss, concurrency, retries, service availability, CPU and memory pressure, database state, and execution timing.

Module 2: Multi-Source Failure Observation Engine

Collects application logs, distributed traces, system metrics, database events, request histories, and application-level correctness invariants.

It converts every execution into a structured experimental record containing the tested conditions, observed behaviour, failure status, and supporting evidence.

Module 3: Uncertainty-Guided Autonomous Experiment Planner

Maintains a model of the relationship between experimental conditions and failure probability.

It estimates uncertain regions, selects the most informative next experiment, balances exploration against failure-focused exploitation, and continuously updates its beliefs using new execution results.

This is the primary AI/ML module.

Module 4: Failure-Boundary Discovery and Trigger Minimization Engine

Identifies regions where the system transitions between correct and failed behaviour.

After discovering a failure, it determines which conditions are necessary, removes irrelevant factors, narrows parameter ranges, and calculates the reliability of the minimized trigger.

Module 5: Autonomous Validation, Regression Generation and Research Dashboard

Repeatedly validates the discovered trigger, measures reproduction probability, generates a deterministic regression test where possible, and presents the complete experimental evidence.

The dashboard visualizes explored configurations, failure regions, uncertainty, experiment history, trigger minimization, and baseline comparisons.

Evaluation and Baselines

The proposed framework will be evaluated using software systems containing controlled but initially hidden intermittent failures.

The main comparison methods will be:

- Random Search;
- Grid or Exhaustive Search where computationally feasible;
- Pairwise or Combinatorial Testing;
- Optimization-Based Search;
- Proposed Uncertainty-Guided Autonomous Experimentation.

Trigger minimization will additionally be compared with a conventional reduction or delta-debugging-style baseline where applicable.

The primary evaluation metrics will include:

- number of executions required for first failure discovery;
- time to first failure discovery;
- total diagnostic executions;
- failure discovery rate;
- trigger size;
- trigger range precision;
- reproduction reliability;
- false causal factor inclusion;
- computational overhead;
- experiment efficiency.

Ablation studies will evaluate whether uncertainty estimation, adaptive experiment selection, failure-boundary exploration, and reliability-aware minimization each contribute meaningfully to the final performance.

Expected Results / Impact

The expected result is a framework capable of discovering intermittent multi-factor failure triggers using fewer executions than random and non-adaptive search strategies.

A successful final output may resemble:

Minimal Failure Trigger

Payment API latency: 1.8–2.2 seconds

Concurrent workers: ≥ 2

Retry occurs before transaction commit

Idempotency protection: absent

Reproduction reliability: 96%

The research is expected to demonstrate that software failure diagnosis can be formulated as an adaptive sequential experimentation problem rather than a purely manual debugging or brute-force testing process.

The practical impact would be reduced debugging time, improved reproducibility of rare production failures, automated generation of evidence-backed failure reports, and creation of regression tests for failures that were previously difficult to reproduce.

Keywords

Intermittent Software Failures, Autonomous Experimentation, Software Testing, Fault Injection, Active Learning, Bayesian Optimization, Uncertainty Estimation, Failure Boundary Discovery, Trigger Minimization, Distributed Systems, Automated Debugging, Search-Based Software Engineering

Title

FORGET-TS: A Model-State-Aware Adaptive Replay Framework for Mitigating Catastrophic Forgetting in Continual Time-Series Forecasting

Background

Time-series forecasting models are increasingly expected to operate in dynamic environments where new data, domains, and distribution patterns arrive continuously. A forecasting model may initially learn electricity demand patterns, later adapt to weather data, then traffic, air quality, or other evolving temporal domains.

Traditional machine learning assumes that all training data are available simultaneously. Real-world systems often violate this assumption. New data arrive sequentially, and repeatedly retraining a complete model using all historical data is computationally expensive and may be impossible because of storage, privacy, or data-retention constraints.

Continual learning attempts to solve this problem by allowing a model to learn new tasks while retaining previously acquired knowledge. However, neural networks suffer from **catastrophic forgetting**: adapting the model to new data can overwrite parameters and representations required for earlier tasks.

For example:

Electricity → Weather → Traffic → Air Quality

After learning the latest task, the model may forecast traffic accurately while its earlier electricity-forecasting capability has significantly degraded. This creates the need for a forecasting system that can continuously adapt without repeatedly forgetting previously learned temporal patterns.

Problem

Existing continual-learning systems frequently protect historical knowledge through replay. Historical samples are stored in memory and periodically mixed with new training data.

However, replay introduces three fundamental questions:

1. **When is replay actually necessary?**
2. **Which previously learned knowledge is currently being threatened?**
3. **Which historical samples are most useful for correcting that specific forgetting?**

Fixed replay wastes computation by revisiting historical data even when the model remains stable. Random replay may retrieve samples unrelated to the knowledge currently being damaged. Storing and repeatedly processing large amounts of historical data also increases memory and computational cost.

The central research problem is therefore:

How can a continually adapting time-series forecasting model detect when and what previously acquired knowledge is being forgotten, and dynamically perform targeted corrective replay while minimizing memory and computational cost?

This is the central problem formulation defined for FORGET-TS.

Existing Limitations

Current approaches have several limitations.

Naive sequential fine-tuning allows the model to learn new tasks but provides no protection against catastrophic forgetting.

Fixed-interval replay performs replay after a predefined number of training steps, regardless of whether forgetting is occurring.

Continuous random replay repeatedly mixes historical samples with new data but may waste computation and retrieve irrelevant historical examples.

Large replay buffers improve access to historical knowledge but create increasing memory and storage requirements.

Performance-only forgetting detection usually discovers forgetting only after historical task performance has already degraded.

Consequently, existing replay strategies are often reactive, computationally inefficient, and insufficiently selective. They generally do not use multiple signals from the model's own internal state to determine whether forgetting is emerging, which historical knowledge is threatened, and how much corrective replay is required.

Research Gap

Most replay-based continual-learning systems follow predetermined or broadly reactive strategies such as:

Replay every N steps

or:

Randomly mix a fixed percentage of historical samples with every new batch

These strategies do not jointly solve the following problems:

- detecting whether replay is currently required;
- identifying the historical task or domain at risk;
- selecting historical samples relevant to that specific forgetting event;

- dynamically determining the required replay budget.

FORGET-TS investigates whether changes in the model's own internal state can provide useful early signals of catastrophic forgetting.

The proposed research direction is:

Observe model state → Estimate forgetting risk → Identify threatened knowledge → Retrieve relevant memories → Perform corrective replay

The precise novelty is not catastrophic forgetting, adaptive replay, representation drift, or gradient conflict individually. Those concepts already exist. The research contribution must come from the specific mechanism that combines model-state signals to control **when, what, and how much to replay** in continual time-series forecasting.

The primary research question is:

Can internal model-state signals be used to predict or detect catastrophic forgetting and dynamically control targeted memory replay in continual time-series forecasting more efficiently than fixed and random replay strategies?

Proposed Solution

FORGET-TS is a continual-learning framework built around a pretrained or pre-initialized time-series forecasting model.

The complete workflow is:

Sequential Data Stream

↓

Time-Series Forecasting Model with PEFT

↓

Internal Model-State Monitoring

↓

Forgetting-Risk Estimation

↓

Threatened-Domain Identification

↓

Adaptive Memory Retrieval

↓

Corrective Replay

↓

Continuous Evaluation

When a new task or domain arrives, the forecasting model adapts using parameter-efficient fine-tuning.

During training, FORGET-TS continuously monitors indicators of model change.

If the model remains stable:

No replay is performed.

If the model shows evidence that previously acquired knowledge is being threatened:

The system identifies the affected historical domain, retrieves relevant historical samples, and performs targeted corrective replay.

The goal is to intervene only when necessary rather than continuously replaying historical data.

Novel Contribution

The primary proposed contribution is a **model-state-aware adaptive replay controller for continual time-series forecasting**.

The framework combines four mechanisms.

1. Multi-Signal Forgetting Detection

Instead of relying exclusively on historical validation loss, the system monitors multiple candidate indicators:

- parameter drift;
- representation drift;
- gradient conflict;
- historical performance degradation.

The hypothesis is that the combination of these signals can provide a more reliable estimate of forgetting risk than any single signal.

2. Threatened-Knowledge Identification

The system does not merely determine that forgetting is occurring. It attempts to identify **which previous task or domain is currently at risk**.

For example:

Electricity: 12% forgetting risk

Weather: 78% forgetting risk

Traffic: 24% forgetting risk

Replay can then focus on weather knowledge rather than randomly sampling the entire historical memory.

3. Event-Triggered Corrective Replay

Replay is treated as an intervention rather than a permanent training process.

The model continues normal adaptation while forgetting risk remains below a threshold. Corrective replay is triggered only when the estimated risk becomes significant.

4. Targeted and Budget-Aware Memory Retrieval

Once threatened knowledge is identified, the framework retrieves high-value historical samples associated with that knowledge and dynamically determines how much replay is necessary.

The intended contribution is therefore the closed-loop mechanism:

Model-state observation → **forgetting-risk estimation** →
threatened-domain identification → **targeted sample selection** → **dynamic corrective replay**

The project hypothesis is that this mechanism can achieve a better **forgetting–compute trade-off** than fixed or random replay.

Methodology

The project will construct a controlled continual-learning environment containing sequential time-series tasks:

$T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow \dots \rightarrow T_\square$

Possible domains include compatible forecasting tasks derived from electricity, weather, traffic, and air-quality datasets. The task sequence must be methodologically defensible; unrelated datasets should not be combined merely to increase the number of domains.

A forecasting model is first trained or adapted on the initial task. A constrained episodic memory stores representative historical samples.

When the next task arrives, the model adapts using parameter-efficient fine-tuning techniques such as:

- LoRA;

- adapter layers;
- selective layer unfreezing.

During adaptation, the framework periodically calculates model-state indicators.

Parameter Drift

The system measures changes in selected trainable parameters relative to an earlier stable checkpoint:

$$D_{\theta} = \|\theta_{\text{current}} - \theta_{\text{reference}}\|_2$$

Possible measurements include L1 distance, L2 distance, cosine distance, layer-wise drift, and LoRA update magnitude.

Representation Drift

Historical anchor samples are passed through the current model, and their internal representations are compared with earlier representations:

$$D_r = 1 - \text{cosine_similarity}(\text{Hold}, \text{Hcurrent})$$

Possible techniques include cosine distance, Centered Kernel Alignment, Maximum Mean Discrepancy, and layer-wise feature distance.

Gradient Conflict

The system compares the learning direction produced by new-domain samples with gradients associated with historical knowledge.

If:

$$\cos(\mathbf{g}_{\text{new}}, \mathbf{g}_{\text{old}}) < 0$$

the learning directions conflict, suggesting that improvement on the new task may damage earlier knowledge.

These model-state indicators are combined into a forgetting-risk estimate.

If risk remains below the intervention threshold, normal training continues.

If risk exceeds the threshold, the system:

1. identifies the threatened historical task;
2. queries the constrained memory buffer;
3. ranks historical samples by relevance or corrective value;
4. determines a replay budget;
5. performs targeted corrective replay;
6. measures whether forgetting risk decreases.

This process continues throughout sequential training.

Core Functionalities / Modules

Module 1: Continual Time-Series Data Pipeline

Creates controlled sequential task streams.

The module handles:

- dataset ingestion;
- timestamp validation;
- missing-value processing;
- resampling;
- normalization;
- sliding-window generation;
- train-validation-test splitting;
- sequential task streaming;
- metadata management;
- data-leakage prevention.

The output is a standardized continual-learning sequence:

$D_1 \rightarrow D_2 \rightarrow D_3 \rightarrow D_4 \rightarrow D_5$

Module 2: PEFT-Based Forecasting Engine

Contains the primary forecasting model and controls sequential adaptation.

Candidate backbone families include:

- Chronos;
- Moirai;
- Tiny Time Mixers;
- PatchTST.

The final project should use one primary backbone and, if computationally feasible, a second backbone for generalization experiments rather than attempting to train every available model.

This module:

- loads the forecasting backbone;
- attaches PEFT components;
- freezes selected parameters;
- adapts sequentially;
- stores checkpoints;
- exposes model parameters;

- extracts hidden representations;
- exposes gradients for drift analysis.

Module 3: Multi-Signal Forgetting Detection Engine

This is the main intelligence layer.

It monitors:

- parameter drift;
- representation drift;
- gradient conflict;
- historical performance degradation.

The module combines these indicators to estimate task-specific forgetting risk and determines whether intervention is required.

Module 4: Adaptive Memory and Targeted Replay Engine

Maintains a constrained episodic memory.

It:

- stores representative historical samples;
- identifies threatened domains;
- ranks historical samples;
- selects high-value corrective examples;
- determines the replay budget;
- triggers replay only when necessary;
- measures whether intervention reduced forgetting.

Module 5: Experimental Evaluation and Explainability Platform

Runs research experiments and presents the complete behaviour of the framework.

It provides:

- live training monitoring;
- model-state visualization;
- forgetting-risk scores;
- replay-event timelines;
- baseline comparisons;
- forgetting curves;
- replay-cost graphs;
- memory-usage analysis;
- trigger statistics;
- automatic experimental reports.

The five-module structure is explicitly defined in the project design.

Evaluation and Baselines

FORGET-TS will be compared against the following methods:

Baseline 1: Independent Training

A separate model is trained for each task. This provides an approximate task-specific upper reference.

Baseline 2: Naive Sequential Fine-Tuning

The model learns:

Task 1 → Task 2 → Task 3 → Task 4 → Task 5

without forgetting protection.

This establishes the severity of catastrophic forgetting.

Baseline 3: Fixed-Interval Replay

Historical samples are replayed after a predefined number of training steps.

This tests whether adaptive triggering is better than fixed scheduling.

Baseline 4: Random Continual Replay

Random historical samples are continuously mixed with new training data.

This tests whether targeted retrieval is more efficient than random replay.

Baseline 5: FORGET-TS

Uses:

- model-state monitoring;
- forgetting-risk estimation;
- adaptive triggering;
- threatened-domain identification;
- targeted memory retrieval;
- PEFT-based adaptation.

The primary evaluation metrics will include:

Forecasting performance

- MAE;
- MSE;
- RMSE;
- MASE;
- sMAPE.

Continual-learning performance

- average forgetting;
- backward transfer;
- average final performance.

Replay efficiency

- number of replay interventions;
- total historical samples processed;
- replay overhead.

Resource efficiency

- memory consumption;
- training time;
- GPU hours;
- total training steps.

Trigger quality

- precision;
- recall;
- F1-score;
- AUROC;
- false-trigger rate;
- missed-forgetting rate.

The evaluation should also include:

- multiple random seeds;
- multiple task orders;
- different memory budgets;
- ablation studies;
- statistical significance testing;
- validation on a second forecasting backbone if computationally feasible.

Expected Results / Impact

The expected result is a continual-learning framework that retains historical forecasting capabilities while requiring fewer replay interventions and fewer historical samples than conventional replay strategies.

The intended experimental pattern is:

Method	Forgetting	Replay Usage	Compute Cost
Naive Sequential Training	High	None	Low

Fixed Replay	Low	Very High	High
Random Replay	Medium	Medium–High	Medium
FORGET-TS	Low	Selective	Medium–Low

The exact numerical results cannot be predicted before experimentation.

The target research conclusion is:

FORGET-TS achieves comparable or lower catastrophic forgetting than conventional replay approaches while requiring fewer replay interventions and fewer historical samples.

The broader impact is a more efficient form of continual adaptation for forecasting systems operating under changing environments and limited memory or computational budgets.

The project is classified in the source design as very high difficulty, flagship MCA level, high AI/ML depth, high research potential subject to novelty verification, and approximately a 9–12 month implementation effort.

Keywords

Continual Learning, Catastrophic Forgetting, Time-Series Forecasting, Adaptive Replay, Targeted Memory Replay, Parameter-Efficient Fine-Tuning, Model-State Monitoring, Representation Drift, Parameter Drift, Gradient Conflict, Episodic Memory, Foundation Models, Concept Drift, Sequential Learning

Title

Task-Agnostic Hallucination Probe: Adversarial Activation Alignment for Cross-Task Hallucination Detection in Large Language Models

Background

Large Language Models can generate fluent and convincing responses that contain unsupported, fabricated, or factually incorrect information. This behaviour, commonly referred to as hallucination, limits the reliability of LLMs in knowledge-intensive and high-stakes applications.

Most hallucination-detection approaches analyse the model's external output using factual verification, retrieval systems, self-consistency, uncertainty scores, or secondary evaluator models. Another research direction investigates the model's internal hidden representations. The underlying hypothesis is that internal activations may contain signals indicating whether a generated answer is truthful or hallucinated, even when the final text appears equally convincing.

A lightweight classifier, commonly called a **probe**, can be trained on hidden activations extracted from selected LLM layers:

Prompt and Response → LLM Hidden Activations → Probe → Hallucination Probability

However, a probe trained on one task may learn task-specific activation patterns rather than a general representation of hallucination.

For example:

Training task: Question Answering

↓

High hallucination-detection accuracy

but:

Test task: Summarization

↓

Significant performance degradation

This creates a cross-task generalization problem for activation-based hallucination detection.

Problem

Different LLM tasks generate different internal activation distributions.

Question answering, summarization, dialogue, reasoning, and long-form generation differ in:

- prompt structure;
- response length;
- token distribution;
- semantic complexity;
- attention patterns;
- layer-wise activation geometry.

A hallucination probe trained on one task can therefore exploit features that correlate with the task rather than features genuinely associated with hallucination.

The central problem is:

How can an activation-based hallucination detector learn representations that identify hallucinations while remaining invariant to the task from which the activations originate?

The challenge is not merely to classify hallucinated and non-hallucinated responses within a known dataset. The harder requirement is:

Train on known tasks and detect hallucinations on an unseen task without retraining the detector for that task.

Existing Limitations

Existing hallucination-detection methods have several limitations.

Output-based factual verification requires external evidence and may fail when reliable reference documents are unavailable.

Retrieval-based verification depends on the quality and coverage of the retrieval corpus.

LLM-as-a-judge methods introduce additional computational cost and may themselves produce incorrect evaluations.

Confidence and token-probability methods are unreliable because LLMs can assign high probability to incorrect outputs.

Standard activation probes may achieve strong in-domain results but degrade under cross-task distribution shift.

A conventional probe learns:

Activation → Hallucination label

but may unintentionally encode:

Activation → Task identity

If task identity and hallucination labels are correlated in the training data, the classifier can achieve apparently strong benchmark performance without learning a genuinely transferable hallucination representation.

Consequently, high in-domain accuracy does not prove that an activation probe has learned a task-independent signal of hallucination.

Research Gap

The core research gap is the lack of reliable cross-task generalization in white-box activation-based hallucination detection.

Most standard probe training optimizes only the hallucination-classification objective:

Minimize hallucination prediction error

This does not explicitly prevent the learned representation from containing task-specific information.

The proposed research investigates a different objective:

Can an activation probe simultaneously learn to predict hallucinations while being explicitly prevented from learning which task produced the activation?

The intended representation should satisfy:

High information about hallucination status

while maintaining:

Low information about task identity

The research therefore investigates whether adversarial task-invariance can reduce covariate shift in latent activation space and improve zero-shot hallucination detection on unseen generation tasks.

The central research question is:

Can adversarial alignment of LLM activation representations improve cross-task zero-shot hallucination detection compared with conventional activation probes?

Proposed Solution

The proposed system is a task-agnostic hallucination-detection framework that analyses internal hidden states extracted from a frozen LLM.

The complete workflow is:

Prompt and Generated Response

↓

Frozen LLM

↓

Layer-Wise Hidden-State Extraction

↓

Activation Pooling and Normalization

↓

Probe Encoder

↓

Task-Invariant Latent Representation

↓

Hallucination Classifier

During training, the probe encoder is connected to two competing prediction heads:

Probe Encoder → Hallucination Classifier

and:

Probe Encoder → Gradient Reversal Layer → Task Discriminator

The hallucination classifier attempts to determine whether the response is hallucinated.

The task discriminator attempts to determine whether the activation originated from question answering, summarization, dialogue, or another task.

The Gradient Reversal Layer reverses the task-discriminator gradient before it reaches the probe encoder.

Therefore:

- the hallucination head pushes the encoder to preserve hallucination-related information;
- the task discriminator pushes the encoder to remove task-identifying information.

The intended result is a latent representation that captures features useful for hallucination detection but generalizes across task boundaries.

Novel Contribution

The primary proposed contribution is an **adversarially aligned activation probe for task-invariant hallucination detection**.

The novelty consists of four connected mechanisms.

1. Cross-Task Activation Representation

Instead of building a separate detector for each task, the system learns a common latent space from hidden activations collected across heterogeneous LLM tasks.

2. Adversarial Task-Invariance

A task discriminator attempts to recover the source task from the learned representation.

The probe encoder is simultaneously optimized to prevent successful task classification.

This creates the adversarial objective:

Learn hallucination signal

while:

Suppressing task identity

3. Gradient-Reversal-Based Activation Alignment

A Gradient Reversal Layer enables end-to-end adversarial training.

During the forward pass, the layer behaves normally.

During backpropagation:

Gradient $\rightarrow -\lambda \times \text{Gradient}$

The encoder is therefore penalized when its representation makes task identity easy to predict.

4. Strict Unseen-Task Evaluation

The framework is evaluated using leave-one-task-out validation.

For example:

Train: Question Answering + Dialogue + Reasoning

Test: Summarization

The unseen task is not used to train the probe.

This prevents the project from claiming task agnosticism based only on random train-test splits from the same task distribution.

The proposed contribution is therefore:

A probe that explicitly optimizes hallucination discriminability and task invariance simultaneously, evaluated under strict cross-task zero-shot transfer.

Methodology

The project begins with a selected open-weight LLM. A model small enough for the available hardware should be preferred over an oversized model that prevents rigorous experimentation.

The LLM remains frozen.

For every labelled example, the system records:

Input prompt

Generated or provided response

Hallucination label

Task label

The example is passed through the LLM, and hidden states are extracted from selected layers.

For layer l :

$$\mathbf{H}^l \in \mathbb{R}^{T \times D}$$

where:

- T = sequence length;
- D = hidden dimension.

Because different examples contain different numbers of tokens, the activation tensor must be transformed into a fixed-dimensional representation.

Candidate strategies include:

- mean pooling;
- last-token pooling;
- attention-weighted pooling;
- statistical pooling;
- learned token aggregation.

The resulting representation is passed through a probe encoder:

$$\mathbf{z} = \mathbf{E}(\mathbf{H})$$

The latent representation $\mathbf{z}$ is sent to two heads.

Hallucination Classification Head

Predicts:

$$\hat{\mathbf{y}}_{\square} = \mathbf{C}_{\square}(\mathbf{z})$$

The hallucination loss is:

$$\mathbf{L}_{\square} = \text{Binary Cross-Entropy}(\mathbf{y}_{\square}, \hat{\mathbf{y}}_{\square})$$

Task Discriminator

Predicts:

$$\hat{\mathbf{y}}_{\square} = \mathbf{C}_{\square}(\text{GRL}(\mathbf{z}))$$

The task loss is:

$$\mathbf{L}_{\square} = \text{Cross-Entropy}(\mathbf{y}_{\square}, \hat{\mathbf{y}}_{\square})$$

The probe encoder is optimized to improve hallucination classification while confusing the task discriminator.

Conceptually:

$$\mathbf{L} = \mathbf{L}_{\square} - \lambda \mathbf{L}_{\square}$$

where λ controls the strength of task-invariance pressure.

The training procedure should compare multiple values of λ because excessive adversarial alignment may remove information that is useful for hallucination detection.

The complete experimental protocol is:

1. collect labelled examples from multiple tasks;
2. extract hidden activations from selected LLM layers;
3. normalize and pool variable-length activation tensors;
4. train the standard probe baseline;
5. train the adversarially aligned probe;
6. hold out one complete task;
7. evaluate both methods on that unseen task;
8. repeat for every task;
9. perform statistical and ablation analysis.

Core Functionalities / Modules

Module 1: Multi-Task Dataset and Activation Extraction Pipeline

Creates the experimental dataset and extracts hidden states from the frozen LLM.

The module handles:

- multi-task dataset ingestion;
- hallucination-label normalization;
- task-label assignment;
- prompt formatting;
- LLM inference;
- layer-wise activation capture;
- activation caching;
- experiment metadata management.

The output is:

Activation + Hallucination Label + Task Label

Module 2: Activation Pooling and Representation Standardization Engine

Transforms variable-length hidden-state tensors into comparable fixed-dimensional representations.

The module performs:

- token-level pooling;
- tensor normalization;
- layer selection;
- activation scaling;
- dimensionality reduction where necessary;
- representation caching.

Multiple pooling methods can be compared experimentally.

Module 3: Hallucination Probe Encoder

Learns a compact representation from the standardized LLM activations.

Candidate architectures include:

- multilayer perceptron;
- 1D-CNN;
- lightweight Transformer encoder.

The architecture should remain significantly smaller than the underlying LLM so that the research studies the information already present in the activations rather than training another large language model.

Module 4: Adversarial Task-Alignment Engine

This is the primary research module.

It contains:

- Gradient Reversal Layer;
- task discriminator;
- adversarial loss controller;
- task-invariance strength scheduling;
- multi-objective training loop.

The module attempts to remove task-specific information while retaining hallucination-related information.

Module 5: Cross-Task Evaluation and Explainability Platform

Runs strict unseen-task experiments and compares the proposed method against baseline probes.

The module provides:

- leave-one-task-out evaluation;
- layer-wise performance analysis;
- latent-space visualization;
- task-separability analysis;
- calibration analysis;
- statistical significance testing;
- ablation studies;
- experiment dashboard.

The five-module architecture follows the original project design: activation extraction, representation standardization, probe encoding, adversarial task discrimination and strict cross-task validation.

Evaluation and Baselines

The project should use a strict **leave-one-task-out** experimental protocol.

For N tasks:

Train on $T_1, T_2, \dots, T_{N-1}$

Test on unseen T_N

The experiment is repeated until every task has served as the unseen target.

The primary baselines should be:

Baseline 1: Output Probability / Confidence Method

Uses model confidence or token-level uncertainty without an activation probe.

Baseline 2: Standard Linear Probe

A simple classifier trained directly on pooled activations.

Baseline 3: Non-Adversarial Neural Probe

Uses the same probe encoder as the proposed model but without the task discriminator or Gradient Reversal Layer.

Baseline 4: Domain-Specific Probe

Trains and evaluates within the same task distribution. This acts as an approximate upper reference but is not task-agnostic.

Baseline 5: Proposed Task-Agnostic Probe

Uses:

- multi-task activation extraction;
- probe encoder;
- Gradient Reversal Layer;
- adversarial task discriminator;
- cross-task zero-shot evaluation.

The primary metrics should include:

Hallucination Detection

- Accuracy;
- Precision;
- Recall;
- F1-score;
- AUROC;
- AUPRC.

Cross-Task Generalization

- average unseen-task performance;
- worst-task performance;
- transfer gap between in-domain and unseen-task results.

Task Invariance

- task-discriminator accuracy;
- task separability of latent representations.

A successful task-invariant representation should make task classification difficult while preserving strong hallucination classification.

Calibration

- Expected Calibration Error;
- Brier Score.

Statistical Analysis

- McNemar's test;
- confidence intervals;
- multiple random seeds.

The essential ablation studies should remove or vary:

- Gradient Reversal Layer;
- task discriminator;
- adversarial loss weight;
- selected LLM layer;
- pooling strategy;
- number of source tasks.

Expected Results / Impact

The expected result is an activation-based hallucination detector that generalizes better to unseen tasks than conventional probes.

The intended result pattern is:

Method	In-Domain Detection	Unseen-Task Detection	Task Dependence
Linear Probe	Moderate	Low	High
Neural Probe	High	Moderate	High
Proposed Adversarial Probe	High	Higher	Low

The exact numerical results cannot be predicted before experimentation.

The target research conclusion is:

Adversarial suppression of task-specific information improves the cross-task generalization of activation-based hallucination detectors without substantially reducing their ability to identify hallucinations.

If successful, the project would contribute toward hallucination detectors that do not require separate retraining for every new LLM application.

The broader impact includes:

- more transferable LLM reliability monitoring;
- reduced dependence on task-specific hallucination datasets;
- improved understanding of whether hallucination-related signals exist in internal model representations;
- lower-cost safety monitoring through lightweight probes;
- stronger evaluation standards for claims of task-agnostic hallucination detection.

Keywords

Large Language Models, Hallucination Detection, Activation Probing, Hidden Representations, Task-Invariant Learning, Domain Generalization, Adversarial Learning, Gradient Reversal Layer, Cross-Task Transfer, Zero-Shot Generalization, White-Box LLM Safety, Representation Alignment, Uncertainty, LLM Reliability

Title

Active Outcome Verification for AI Agents: An Uncertainty-Guided Framework for Detecting Silent Semantic Failures through Adaptive Evidence Acquisition

Background

AI agents are increasingly capable of executing multi-step workflows involving APIs, databases, web applications, external services, business systems, and other software tools.

A typical agent may:

Receive Goal → Plan Actions → Call Tools → Observe Responses → Report Completion

For example:

Goal: Refund a customer and update the CRM.

The agent may execute:

Refund API → HTTP 200

CRM update → Success

Agent conclusion → Task completed

However, successful execution does not necessarily mean that the user's actual goal was achieved.

The refund API may only have accepted the request while the transaction later fails. The CRM may record the customer as refunded even though no money reaches the customer. Every tool call can therefore succeed technically while the overall real-world outcome remains incorrect.

This creates an important distinction:

Execution Success ≠ Goal Success

As AI agents become more autonomous, determining whether an agent actually achieved its intended outcome becomes a fundamental reliability problem.

Problem

Current AI agents commonly determine success from immediate execution signals such as:

- HTTP status codes;
- API responses;
- database updates;
- tool-return messages;

- workflow completion;
- agent-generated reasoning.

These signals prove that actions were executed, but they may not prove that the intended real-world outcome occurred.

Consider:

Goal: Cancel subscription and stop future billing.

The agent performs:

Cancellation API → Success

but:

Billing system → Pending renewal remains active

or:

Goal: Send an important document to a customer.

The agent observes:

Email API → Accepted

but:

Actual outcome → Delivery failed later

The central problem is therefore:

How can an AI agent determine whether its intended goal was actually achieved when available evidence is incomplete, conflicting, costly to obtain, and may change over time?

A second problem immediately follows:

What should the agent verify next when current evidence is insufficient to prove success?

The original project concept correctly identifies this deeper problem as active verification under incomplete, conflicting, costly and temporally changing evidence.

Existing Limitations

Current agent-verification approaches have several limitations.

Tool-Success Checking

The simplest approach trusts tool responses.

For example:

API returned 200 → Success

This is unreliable because an API may confirm that a request was accepted rather than that the intended outcome was completed.

Static Postcondition Checking

A developer manually defines conditions such as:

```
refund.status == "completed"
```

This works for predictable workflows but requires exhaustive manual rules and may fail when outcomes depend on multiple systems or delayed state changes.

Execution-Trace Analysis

The agent's action sequence is analysed after completion.

However, a logically valid execution trace does not prove that external systems reached the intended final state.

LLM-as-a-Judge

Another LLM evaluates whether the agent appears to have succeeded.

This can judge semantic coherence but usually sees the same limited evidence as the original agent. It may confidently approve an outcome that was never independently verified.

Exhaustive Verification

The system checks every possible source of evidence.

This may provide high confidence but is expensive, slow, and impractical.

For example, verifying a refund could involve:

- payment provider status;
- bank settlement status;
- internal ledger;
- CRM record;
- customer notification;
- delayed rechecking.

Performing every check for every task creates excessive cost and latency.

The core limitation is that existing approaches often either **verify too little** or **verify everything**.

They do not efficiently answer:

Given what is already known, which verification action would reduce uncertainty the most?

Research Gap

The research gap is not simply the absence of outcome checking.

Postconditions, assertions, workflow validation, monitoring, and agent evaluation already exist.

The deeper gap is **adaptive outcome verification**.

An intelligent verification system should determine:

1. what evidence is currently available;
2. how reliable each evidence source is;
3. whether the evidence conflicts;
4. what remains uncertain;
5. which verification action should be performed next;
6. whether the value of additional evidence justifies its cost;
7. when sufficient evidence exists to allow the agent to claim success.

The research problem can therefore be formulated as:

Current Evidence

↓

Estimate Outcome Uncertainty

↓

Identify Missing Information

↓

Select Most Valuable Verification Action

↓

Acquire New Evidence

↓

Update Belief

↓

Success Sufficiently Proven?

If no:

Continue Verification

If yes:

Allow Completion

The central research question is:

Can uncertainty-guided active evidence acquisition detect silent semantic failures at lower verification cost than fixed checks, exhaustive verification, and trace-based LLM judges?

This is the research question defined in the original concept.

Proposed Solution

The proposed system is an independent outcome-verification layer for AI agents.

Instead of trusting the agent's declaration of success, the framework receives:

- the original user goal;
- the agent's execution trace;
- tool outputs;
- available system states;
- possible verification actions.

The framework first converts the user's goal into a structured set of expected outcomes.

For example:

Goal: Refund the customer and update the CRM.

The system derives:

Expected Outcome 1: Refund transaction initiated.

Expected Outcome 2: Payment provider confirms completion.

Expected Outcome 3: Customer balance reflects the refund.

Expected Outcome 4: CRM status matches the actual payment state.

The system then gathers available evidence and estimates the probability that each expected outcome has been achieved.

If evidence is insufficient, it does not immediately declare failure or execute every possible check.

Instead, it asks:

Which available verification action would reduce the most uncertainty for the lowest cost?

The complete workflow is:

User Goal



Agent Execution



Expected Outcome Derivation



Evidence Collection



Outcome Belief Estimation



Uncertainty Analysis



Next Verification Action Selection



Independent Evidence Acquisition



Belief Update



Success Sufficiently Proven?

If yes:

Verified Success

If no:

Continue Verification / Diagnose / Recover / Escalate

This follows the core process defined in the original idea: derive expected outcomes, gather evidence, estimate uncertainty, choose the next verification action, collect independent evidence, and determine whether success is sufficiently proven.

Novel Contribution

The primary proposed contribution is an **uncertainty-guided active evidence acquisition framework for verifying whether AI agents actually achieve their intended outcomes**.

The novelty consists of four connected mechanisms.

1. Goal-to-Outcome Decomposition

The system distinguishes between:

Actions performed

and:

Outcomes that must become true

For example:

Action:

Call refund API.

Outcome:

Customer actually receives the refunded amount.

This prevents successful tool execution from being treated as sufficient proof of goal completion.

2. Probabilistic Evidence Aggregation

Evidence sources may have different levels of reliability.

For example:

Agent statement: Weak evidence

API acceptance response: Moderate evidence

Payment-provider completion status: Strong evidence

Independent ledger confirmation: Strong evidence

The system combines multiple evidence sources to estimate confidence in the actual outcome.

3. Uncertainty-Guided Verification Selection

Instead of following a fixed verification checklist, the system selects the next verification action dynamically.

The decision considers:

- current uncertainty;
- expected information gain;
- evidence reliability;
- verification cost;
- latency;
- redundancy with existing evidence.

The system therefore attempts to maximize:

Verification Value = Expected Uncertainty Reduction / Verification Cost

4. Temporal Outcome Verification

Some outcomes cannot be verified immediately.

For example:

Email accepted now → delivery failure reported later.

or:

Refund initiated now → settlement confirmed hours later.

The framework maintains unresolved outcome states and performs delayed verification when necessary.

The proposed contribution is therefore not merely a success checker.

It is the closed-loop mechanism:

Goal decomposition → probabilistic evidence modelling → uncertainty estimation → active verification selection → temporal belief update → evidence-backed completion decision

Methodology

Each agent task is represented as a goal:

G

The system decomposes the goal into a set of expected outcomes:

$$\mathbf{O} = \{o_1, o_2, \dots, o_n\}$$

For each expected outcome, the system maintains a belief:

$$P(o_i = \text{achieved} \mid \mathbf{E})$$

where $\mathbf{E}$ represents the currently available evidence.

Evidence may include:

- API responses;
- database states;
- external service states;
- transaction records;
- execution traces;
- delayed events;
- independent confirmation sources.

Each evidence source is represented using attributes such as:

- relevance;
- reliability;
- independence;
- timestamp;
- cost of acquisition.

The system initially gathers low-cost available evidence.

It then calculates uncertainty for each expected outcome.

A possible uncertainty measure is entropy:

$$H(o) = -p \log(p) - (1-p) \log(1-p)$$

High entropy indicates that the system is uncertain whether the outcome occurred.

The framework then evaluates candidate verification actions:

$$\mathbf{V} = \{v_1, v_2, \dots, v_n\}$$

For each action, it estimates:

- expected information gain;
- execution cost;
- expected latency;
- evidence reliability.

The next verification action is selected using an acquisition function such as:

$$\text{Utility}(v) = \text{Expected Information Gain}(v) - \lambda \times \text{Cost}(v)$$

or:

$$\text{Utility}(v) = \text{Expected Uncertainty Reduction}(v) / \text{Cost}(v)$$

After executing the selected verification action, new evidence is collected:

$$E \leftarrow E \cup E_{\text{new}}$$

The outcome belief is updated.

The process repeats until one of the following conditions is reached:

Verified Success

Confidence exceeds the required threshold.

Verified Failure

Evidence strongly indicates that the expected outcome did not occur.

Unresolved

Additional verification is possible and valuable.

Escalation

The system cannot obtain sufficient evidence automatically.

The project will experimentally compare adaptive verification against static and non-adaptive strategies.

Core Functionalities / Modules

Module 1: Goal and Expected-Outcome Decomposition Engine

Transforms the user's high-level goal into explicit, verifiable outcome claims.

The module:

- parses the user goal;
- analyses the agent plan;
- separates actions from outcomes;
- generates expected postconditions;
- identifies dependencies between outcomes;
- creates an outcome-verification graph.

Example:

Goal:

Refund customer and update CRM.

Output:

Refund requested
Refund completed
Customer financial state updated
CRM synchronized with actual outcome

Module 2: Multi-Source Evidence Collection and Normalization Engine

Collects evidence from available systems.

Sources may include:

- agent traces;
- API responses;
- databases;
- event streams;
- transaction records;
- external services;
- delayed callbacks.

The module converts heterogeneous observations into structured evidence records containing:

Source → Claim → Reliability → Timestamp → Cost → Independence

Module 3: Probabilistic Outcome Belief and Uncertainty Engine

This is the primary inference layer.

It:

- estimates the probability that each expected outcome occurred;
- combines supporting and conflicting evidence;
- measures uncertainty;
- identifies weakly verified outcomes;
- detects evidence contradictions;
- maintains beliefs over time.

The output may resemble:

Refund initiated: 99% confidence
Refund completed: 58% confidence
CRM synchronized: 97% confidence

The system therefore knows that the actual financial outcome remains insufficiently verified.

Module 4: Active Verification Planning and Evidence Acquisition Engine

This is the primary research module.

It:

- generates candidate verification actions;
- estimates expected information gain;
- estimates verification cost;
- ranks available checks;
- selects the next verification action;
- acquires additional evidence;
- updates the verification strategy after every observation.

The system stops when sufficient evidence exists rather than following a predetermined checklist.

Module 5: Temporal Validation, Recovery and Research Evaluation Platform

Manages outcomes that evolve over time.

It:

- schedules delayed verification;
- detects silent failures;
- initiates recovery where permitted;
- escalates unresolved cases;
- records complete evidence trails;
- compares verification strategies;
- visualizes confidence evolution;
- generates research reports.

The dashboard should display:

- original goal;
- agent actions;
- expected outcomes;
- evidence graph;
- confidence per outcome;
- selected verification actions;
- verification cost;
- final completion decision.

Evaluation and Baselines

The project should be evaluated using controlled agent environments containing known silent semantic failures.

Example domains could include:

- payment and refund workflows;
- order-processing workflows;
- CRM synchronization;
- email and notification delivery;
- file-processing pipelines;
- multi-service business workflows.

The benchmark should inject scenarios where:

- all tools succeed and the outcome succeeds;
- a tool visibly fails;
- all immediate tool calls succeed but the final outcome fails;
- evidence sources conflict;
- failure becomes observable only after a delay.

The primary baselines should be:

Baseline 1: Trust Agent Completion

The task is considered successful whenever the agent reports completion.

Baseline 2: Tool-Status Verification

Success is determined from tool responses and API status codes.

Baseline 3: Fixed Verification Checklist

The same predefined checks are executed for every task.

Baseline 4: Exhaustive Verification

Every available verification action is executed.

This provides a high-cost reference.

Baseline 5: Trace-Based LLM Judge

An LLM examines the agent execution trace and predicts whether the task succeeded.

Baseline 6: Proposed Active Outcome Verification

Uses:

- goal decomposition;
- probabilistic evidence aggregation;
- uncertainty estimation;
- adaptive verification selection;
- temporal outcome tracking.

The primary evaluation metrics should include:

Failure Detection

- silent failure detection rate;
- precision;
- recall;
- F1-score;
- AUROC.

Verification Efficiency

- number of verification actions;
- total verification cost;
- time to verified decision;
- unnecessary verification rate.

Decision Quality

- false-success rate;
- false-failure rate;
- unresolved outcome rate.

Calibration

- Expected Calibration Error;
- Brier Score.

Active Verification Quality

- information gained per verification action;
- uncertainty reduction per unit cost.

The most important metric is the trade-off between:

Silent Failure Detection

and:

Verification Cost

The project succeeds only if it detects failures reliably while requiring fewer checks than exhaustive verification.

Expected Results / Impact

The expected result is an outcome-verification framework that detects silent semantic failures more reliably than trusting agent completion, tool responses, or execution traces while requiring fewer verification actions than exhaustive checking.

The intended experimental pattern is:

Method	Silent Failure Detection	Verification Cost
Trust Agent	Low	Very Low
Tool Status	Low–Medium	Low
Fixed Checks	Medium–High	Medium
Exhaustive Verification	High	Very High
Active Outcome Verification	High	Adaptive

The exact numerical results cannot be predicted before experimentation.

The target research conclusion is:

Uncertainty-guided active evidence acquisition can detect silent semantic failures in AI-agent workflows while requiring fewer verification actions than fixed and exhaustive verification strategies.

The practical impact is significant because autonomous agents should not be allowed to equate successful tool execution with successful goal completion.

A mature version of the framework could operate as an independent reliability layer between AI agents and real-world systems:

AI Agent

↓

Action Execution

↓

Active Outcome Verification

↓

Verified Success / Recovery / Escalation

The original project assessment identifies this as a current, software-only and productizable AI-native direction, while also noting its main risk: agent reliability is becoming a crowded research area, so the novelty must remain specifically focused on uncertainty-guided active evidence acquisition rather than generic agent evaluation.

Keywords

**AI Agents, Outcome Verification, Semantic Failure, Active Evidence Acquisition,
Uncertainty Estimation, Agent Reliability, Goal Verification, Probabilistic Inference,
Information Gain, Adaptive Verification, Tool-Using Agents, Temporal Verification,
Silent Failure Detection, Autonomous Agents, Agent Evaluation**

Title

AuraGuard: A Causality-Guided Self-Healing Microservice Framework Using Spatio-Temporal Graph Neural Networks for Cascading Failure Detection and Automated Remediation

Background

Modern cloud applications are increasingly built using microservice architectures in which an application is divided into multiple independently deployed services. These services communicate through APIs, message queues, databases, and service-to-service network calls.

A typical request may follow a dependency chain such as:

User → API Gateway → Order Service → Payment Service → Inventory Service → Database

The health of one service can therefore affect several other services.

For example:

```
Database latency increases
↓
Inventory Service becomes slow
↓
Order Service requests accumulate
↓
API Gateway latency increases
↓
CPU and memory usage rise across multiple services
```

Monitoring systems may now generate alerts for five different components even though the actual initiating fault originated from only one component.

This creates three connected challenges:

Detection: Is the distributed system entering an abnormal state?

Diagnosis: Which component is the actual root cause rather than merely a downstream victim?

Recovery: What intervention can safely restore the system without creating additional failures?

Traditional threshold-based monitoring is poorly suited to this problem because microservice failures are relational and temporal. The significance of a CPU spike or latency increase depends on where the affected service is located in the dependency graph, which services depend on it, and how the abnormal behaviour propagates over time.

AuraGuard treats a microservice system as a continuously evolving graph and combines graph-based anomaly detection, causal root-cause analysis, and automated remediation.

Problem

Cascading failures in microservice systems are difficult to detect and diagnose because the observable symptoms are distributed across multiple interconnected services.

Consider:

Payment Database

↓

Payment Service

↓

Order Service

↓

API Gateway

A failure begins as increased database latency.

Shortly afterward:

- Payment Service latency increases;
- request queues accumulate;
- Order Service begins timing out;
- retries increase network traffic;
- API Gateway response time rises;
- CPU usage increases across several services.

A conventional monitoring system may report:

Payment Service: High latency
Order Service: High error rate
API Gateway: High response time
Worker Service: High CPU

However, these alerts do not explain:

- which service initiated the cascade;
- which services are only downstream victims;
- how the failure propagated;
- which remediation action should be executed.

The central problem is:

How can a distributed microservice system automatically detect cascading failures, identify their causal origin, and select an effective remediation action using the system's evolving dependency structure and telemetry?

Existing Limitations

Static Threshold-Based Monitoring

Traditional monitoring systems use rules such as:

CPU > 90% → Alert
Latency > 2 seconds → Alert
Error rate > 5% → Alert

These methods analyse individual metrics independently.

A latency of 500 ms may be normal for one service and abnormal for another. More importantly, threshold alerts cannot determine whether an abnormal service is the root cause or merely affected by another failing dependency.

Independent Time-Series Anomaly Detection

Machine learning models can detect unusual CPU, memory, latency, and traffic patterns.

However, analysing each service independently ignores the dependency structure of the system.

The model may know:

Service B is abnormal.

but not:

Service B became abnormal because Service A degraded first.

Static Dependency Graphs

Some observability systems construct service maps from distributed traces.

However, microservice topology is dynamic. Services scale horizontally, containers restart, dependencies change, and traffic patterns evolve.

A static graph may therefore fail to represent the actual runtime system.

Correlation-Based Root-Cause Analysis

Two services becoming abnormal simultaneously does not prove that one caused the other.

For example:

Service A abnormal
Service B abnormal

does not necessarily imply:

Service A → caused → Service B

Both may be affected by a third hidden component.

Rule-Based Auto-Remediation

Existing self-healing systems commonly use predefined rules:

High CPU → Add replica
Service unavailable → Restart container

These rules can execute the wrong intervention.

For example, scaling a service will not solve a downstream database bottleneck and may increase database load further.

Detection Without Closed-Loop Recovery

Many anomaly-detection systems stop after generating an alert.

They do not complete the full operational cycle:

Detect → Diagnose → Intervene → Verify

As a result, human engineers must still interpret the anomaly, identify the root cause, choose a recovery action, and verify whether the intervention worked.

Research Gap

Research exists independently in:

- microservice anomaly detection;
- Graph Neural Networks;
- root-cause analysis;
- causal discovery;
- Kubernetes auto-scaling;

- self-healing systems.

Therefore, the project cannot claim that any of these concepts is individually new.

The research gap investigated by AuraGuard is the closed-loop integration of:

Dynamic service topology

↓

Spatio-temporal anomaly propagation modelling

↓

Causal root-cause isolation

↓

Risk-aware remediation selection

↓

Post-remediation outcome verification

The key unresolved question is whether explicitly modelling both the **graph structure** and **temporal propagation** of anomalies can improve root-cause localization and enable more effective automated remediation than methods that analyse metrics, traces, or correlations independently.

The central research question is:

Can a causality-guided spatio-temporal graph framework detect cascading microservice failures, localize their originating service, and select effective remediation actions more accurately than threshold-based, independent time-series, and correlation-based approaches?

A secondary research question is:

Does verifying the system state after remediation reduce ineffective or harmful automated recovery actions compared with one-shot rule-based self-healing?

Proposed Solution

AuraGuard is an intelligent observability and self-healing framework that continuously models a microservice application as a dynamic spatio-temporal graph.

The complete workflow is:

Microservice System



Metrics + Logs + Distributed Traces



Dynamic Service Dependency Graph



Spatio-Temporal GNN



Anomaly Detection and Propagation Analysis



Causal Root-Cause Discovery



Remediation Candidate Generation



Risk-Aware Action Selection



Kubernetes Remediation



Post-Action Verification

The system represents each microservice as a graph node.

Nodes

Examples:

- API Gateway;
- Order Service;
- Payment Service;
- Inventory Service;
- Database.

Each node contains telemetry features such as:

- CPU usage;
- memory usage;
- request latency;
- error rate;
- request throughput;
- queue length;
- replica count.

Edges

Edges represent runtime dependencies:

Order Service → Payment Service

An edge may contain:

- request count;
- average latency;
- error rate;
- timeout frequency;
- retry count.

The graph changes over time:

$G_1 \rightarrow G_2 \rightarrow G_3 \rightarrow \dots \rightarrow G_n$

A Spatio-Temporal Graph Neural Network learns both:

- how abnormal behaviour propagates between connected services;
- how that behaviour evolves across time.

When a cascading anomaly is detected, the causal analysis engine examines the anomaly window to distinguish the initiating service from downstream symptoms.

The system then selects a remediation action such as:

- restart service;
- scale replicas;
- apply rate limiting;
- isolate unhealthy instance;
- reroute traffic;
- rollback deployment.

After executing the action, AuraGuard observes the system again.

If system health improves:

Remediation validated.

If the failure persists:

Update diagnosis → Select alternative action → Re-evaluate.

Novel Contribution

The primary proposed contribution is a **causality-guided closed-loop self-healing framework that combines dynamic topology modelling, spatio-temporal failure propagation analysis, and outcome-verified remediation.**

1. Dynamic Spatio-Temporal System Representation

AuraGuard does not treat telemetry as independent time series.

The system is represented as:

$$\mathbf{G}_t = (\mathbf{V}_t, \mathbf{E}_t, \mathbf{X}_t)$$

where:

- $\mathbf{V}_t$ = active services at time t ;
- $\mathbf{E}_t$ = runtime dependencies;
- $\mathbf{X}_t$ = telemetry features.

The model therefore learns from both system topology and temporal behaviour.

2. Cascade-Aware Anomaly Detection

Instead of independently asking:

Which services are abnormal?

the system investigates:

How did the abnormal state propagate through the dependency graph?

This allows AuraGuard to distinguish:

Originating anomaly

from:

Propagated anomaly

3. Causal Root-Cause Refinement

A high anomaly score does not automatically indicate root cause.

AuraGuard therefore separates:

Anomaly Detection

from:

Causal Diagnosis

The ST-GNN identifies the abnormal subgraph and propagation pattern. A causal analysis layer then investigates directional relationships during the anomaly window.

4. Risk-Aware Remediation Selection

AuraGuard evaluates multiple possible interventions rather than blindly applying a fixed rule.

A remediation action can be scored using:

$$\text{Action Utility} = \text{Expected Recovery Benefit} - \text{Action Cost} - \text{Operational Risk}$$

5. Closed-Loop Outcome Verification

The system does not assume that an executed Kubernetes command solved the problem.

It verifies:

- anomaly reduction;
- latency recovery;
- error-rate reduction;
- graph stabilization;
- recurrence of the failure.

The proposed contribution is therefore:

**Dynamic topology modelling → cascade detection → causal localization →
remediation selection → outcome verification**

rather than simply “using a GNN for anomaly detection.”

Methodology

The microservice system at time t is represented as a graph:

$$G_t = (V_t, E_t, X_t)$$

For every service node v , a feature vector is constructed:

$$x_v^t = [\text{CPU}, \text{Memory}, \text{Latency}, \text{ErrorRate}, \text{Throughput}, \text{QueueLength}]$$

For every dependency edge:

$e_i^t = [\text{RequestRate}, \text{EdgeLatency}, \text{TimeoutRate}, \text{RetryRate}]$

The system maintains a temporal sequence:

$G_{t-1}, \dots, G_{t-2}, G_{t-1}, G_t$

Spatial Modelling

A graph neural network aggregates information from neighbouring services.

Conceptually:

$h_v = \text{GNN}(x_v, \text{Neighbours}(v))$

The representation of a service therefore depends on both its own telemetry and the behaviour of connected services.

Candidate architectures include:

- Graph Convolutional Networks;
- Graph Attention Networks;
- GraphSAGE.

Temporal Modelling

The temporal component models changes across graph snapshots.

Candidate methods include:

- LSTM or GRU over graph embeddings;
- Temporal Graph Networks;
- Spatio-Temporal Graph Convolutional Networks.

The model outputs:

- service-level anomaly scores;
- system-level anomaly score;
- abnormal subgraph;
- estimated propagation sequence.

Causal Analysis

When an anomaly window is detected, causal discovery is performed on relevant telemetry variables.

Candidate methods include:

- PCMCI;
- Granger-style causal analysis;
- structural causal modelling.

The objective is to estimate a causal structure such as:

Database latency

↓

Payment latency

↓

Order timeouts

↓

Gateway degradation

instead of treating all four symptoms as equivalent.

Remediation Selection

The system generates candidate actions:

$A = \{a_1, a_2, \dots, a_n\}$

Each action is evaluated according to:

- predicted recovery benefit;
- confidence in diagnosis;
- execution cost;
- operational risk.

The selected action is executed through the orchestration layer.

Outcome Verification

After remediation, AuraGuard compares:

Pre-Action State

with:

Post-Action State

The action is considered effective only if the relevant failure indicators improve and the system remains stable during a verification period.

Core Functionalities / Modules

Module 1: Distributed Telemetry Ingestion and Processing Engine

Collects live observability data from the microservice environment.

It handles:

- OpenTelemetry traces;
- CPU and memory metrics;
- request latency;
- error rates;
- throughput;
- logs;
- Kubernetes events.

The data pipeline synchronizes telemetry from different sources and converts it into standardized time windows.

A suitable implementation can use:

- OpenTelemetry;
- Prometheus;
- Kafka;
- Python.

Module 2: Dynamic Microservice Topology Graph Engine

Continuously reconstructs the runtime service dependency graph from distributed traces and communication data.

It:

- discovers services;
- identifies service dependencies;
- creates graph nodes and edges;
- updates changing topology;
- stores temporal graph snapshots;
- attaches telemetry features.

The output is:

$G_1 \rightarrow G_2 \rightarrow \dots \rightarrow G_n$

A graph database such as Neo4j may be used for exploration, although in-memory graph structures may be more efficient for model training.

Module 3: Spatio-Temporal Graph Anomaly and Cascade Detection Engine

This is the primary AI/ML module.

It:

- processes temporal graph sequences;
- learns normal system behaviour;
- calculates service anomaly scores;
- detects abnormal subgraphs;
- models anomaly propagation;
- identifies likely cascade sequences.

The model may be implemented using PyTorch and PyTorch Geometric.

Module 4: Causal Root-Cause Analysis and Remediation Intelligence Engine

This is the primary research and decision layer.

It:

- extracts anomaly windows;
- constructs candidate causal relationships;
- distinguishes causes from downstream symptoms;
- ranks root-cause candidates;
- generates remediation options;
- evaluates action risk and expected benefit;
- selects an intervention.

Candidate causal methods include PCMCI and structural causal models.

Module 5: Automated Self-Healing, Verification and Research Dashboard

Executes approved remediation actions through the Kubernetes API.

It supports:

- service restart;
- horizontal scaling;
- rate limiting;
- unhealthy-instance isolation;
- traffic rerouting;
- rollback.

It then verifies whether the action actually restored system health.

The dashboard displays:

- live microservice topology;
- node-level anomaly scores;
- detected cascade paths;
- ranked root causes;

- causal graphs;
- selected remediation;
- before/after system state;
- baseline comparison;
- experimental metrics.

The original AuraGuard architecture defines the same core progression: telemetry ingestion, dynamic topology modelling, ST-GNN detection, causal inference, and Kubernetes remediation.

Evaluation and Baselines

AuraGuard should be evaluated on a controlled Kubernetes microservice testbed containing deliberately injected failures.

Suitable failure scenarios include:

- network latency;
- packet loss;
- CPU saturation;
- memory pressure;
- database slowdown;
- service crash;
- dependency timeout;
- retry storm;
- overloaded downstream service;
- cascading multi-service failure.

The benchmark must record the true injected root cause so that root-cause predictions can be objectively evaluated.

The primary baselines should be:

Baseline 1: Static Threshold Monitoring

Uses manually configured CPU, memory, latency, and error-rate thresholds.

Baseline 2: Independent Time-Series Anomaly Detection

Analyses each service separately without graph structure.

Baseline 3: Graph-Based Anomaly Detection Without Temporal Modelling

Uses the dependency graph but does not model historical evolution.

Baseline 4: Spatio-Temporal GNN Without Causal Analysis

Detects abnormal services and propagation patterns but selects root causes directly from anomaly scores.

Baseline 5: Correlation-Based Root-Cause Ranking

Ranks services using temporal correlation rather than causal analysis.

Baseline 6: Rule-Based Self-Healing

Uses predefined mappings such as:

High CPU → Scale
Crash → Restart

Baseline 7: Proposed AuraGuard Framework

Uses:

- dynamic topology;
- spatio-temporal graph modelling;
- causal root-cause analysis;
- risk-aware remediation;
- post-action verification.

The primary evaluation metrics should include:

Anomaly Detection

- Precision;
- Recall;
- F1-score;
- AUROC;
- detection latency.

Root-Cause Localization

- Top-1 accuracy;
- Top-3 accuracy;
- Mean Reciprocal Rank;
- localization latency.

Cascade Analysis

- propagation-path accuracy;
- affected-service identification accuracy.

Remediation

- successful recovery rate;
- Mean Time to Recovery;
- unnecessary-action rate;
- harmful-action rate.

System Efficiency

- inference latency;
- telemetry-processing overhead;
- CPU and memory overhead.

The most important ablation studies should remove:

- graph structure;
- temporal modelling;
- causal analysis;
- risk-aware action selection;
- post-remediation verification.

This determines whether each proposed component contributes measurable value.

Expected Results / Impact

The expected result is a self-healing framework capable of detecting cascading microservice failures, identifying their likely causal origin, and selecting effective recovery actions more accurately than isolated monitoring and rule-based remediation.

The intended experimental pattern is:

Method	Cascade Detection	Root-Cause Accuracy	Automated Recovery
Threshold Monitoring	Low	Low	No
Independent ML	Medium	Low	No
Graph Anomaly Detection	High	Medium	No
ST-GNN	High	Medium–High	No
AuraGuard	High	High	Yes

The exact numerical results cannot be predicted before experimentation.

The target research conclusion is:

Combining dynamic service topology, spatio-temporal anomaly propagation modelling, and causal root-cause analysis improves cascading-failure diagnosis, while closed-loop outcome verification makes automated remediation more reliable than one-shot rule-based recovery.

The practical impact includes:

- faster root-cause identification;
- reduced alert overload;
- lower Mean Time to Recovery;
- fewer unnecessary remediation actions;
- improved reliability of distributed applications;
- reduced dependence on manual DevOps intervention.

AuraGuard is particularly suitable as a flagship project because each module is independently demonstrable while contributing to one coherent research problem. It also combines advanced AI/ML with distributed systems, observability, causal inference, and cloud-native engineering rather than attaching unrelated technologies to a single application.

Keywords

Microservices, Spatio-Temporal Graph Neural Networks, Self-Healing Systems, Cascading Failure Detection, Root-Cause Analysis, Causal Inference, Dynamic Graphs, Distributed Systems, Kubernetes, Observability, OpenTelemetry, Fault Injection, Automated Remediation, Graph Neural Networks, Site Reliability Engineering

Title

SwitchSER: Selective Speech Emotion Recognition for Hindi-English Code-Switched Speech with Language-Aware Out-of-Distribution Detection

Background

Speech Emotion Recognition (SER) aims to automatically identify emotional states such as happiness, sadness, anger, frustration, fear, and neutrality from spoken language.

Modern SER systems are commonly trained on monolingual datasets such as English speech corpora. These systems assume that the linguistic and acoustic characteristics encountered during deployment are similar to those observed during training.

This assumption does not hold in multilingual societies such as India.

Bilingual speakers frequently use **code-switching**, where multiple languages are naturally mixed within the same utterance.

For example:

“Yaar, I'm so stressed, kuch samajh nahi aa raha.”

This utterance contains Hindi and English within a single emotional expression. Its emotion cannot be reliably analysed by treating it as purely English or purely Hindi.

Code-switching creates changes in:

- lexical structure;
- phonetic patterns;
- prosody;
- pronunciation;
- language boundaries;
- emotional expression.

A monolingual SER system may therefore receive code-switched speech as an out-of-distribution input.

The major reliability problem is not merely that the model may make an incorrect prediction. The more serious problem is that it may produce the incorrect prediction with high confidence and provide no indication that the input lies outside its reliable operating region.

Therefore:

Unknown Speech Pattern → Confident Emotion Prediction → Silent Failure

SwitchSER investigates whether an SER system can recognise emotions from Hindi-English code-switched speech while also identifying situations in which it should refuse to make an unreliable prediction. The original project formulation identifies this silent high-confidence

failure as the core motivation for moving from conventional SER to selective code-switched SER.

Problem

Most Speech Emotion Recognition systems are trained and evaluated on monolingual speech.

A typical system follows:

Speech

↓

Acoustic Encoder

↓

Emotion Classifier

↓

Forced Emotion Prediction

For every input, the system must output an emotion.

This creates a problem when the input contains an unfamiliar language mixture.

Consider three utterances:

Utterance A

100% English

Utterance B

100% Hindi

Utterance C

50% Hindi + 50% English

These utterances do not necessarily belong to the same input distribution.

A model trained primarily on English speech may correctly recognise emotion in Utterance A but encounter Utterance C as an unfamiliar linguistic-acoustic distribution.

However, conventional SER systems do not explicitly ask:

Is this utterance sufficiently similar to the data on which I learned to make reliable predictions?

Instead, they may output:

Angry — 94% confidence

even when the prediction is unreliable.

The central problem is therefore:

How can a Speech Emotion Recognition system reliably classify emotions in Hindi-English code-switched speech while detecting linguistically out-of-distribution inputs and abstaining when the prediction is unreliable?

A second problem follows:

How should out-of-distribution uncertainty be calculated when the input itself contains different proportions of multiple languages?

The precise source project frames this as a joint problem involving multilingual sub-utterance representation, detection of unfamiliar code-switching patterns, and selective abstention.

Existing Limitations

Monolingual SER Models

Most conventional SER models are trained using a single language.

An English-trained model may learn:

- English phonetics;
- English prosody;
- English lexical patterns;
- English emotional expression.

When Hindi segments appear inside an English utterance, the model may treat them as unfamiliar acoustic patterns rather than meaningful language.

Forced Prediction

Traditional emotion classifiers must always choose a class:

Angry / Happy / Sad / Neutral

Even when the model encounters an unfamiliar input, it still produces a prediction.

There is no:

“I do not have sufficient confidence to classify this utterance reliably.”

Generic Confidence Thresholding

A simple selective classifier may use:

If confidence < 0.6 → Abstain

However, neural-network softmax probabilities are frequently miscalibrated.

A model can be wrong with 95% confidence.

Therefore:

Low Softmax Confidence ≠ Reliable OOD Detection

and:

High Softmax Confidence ≠ Correct Prediction

Generic OOD Detection

Conventional OOD detection usually calculates one global score for the complete utterance.

For example:

Speech → Encoder → Energy Score → OOD / In-Distribution

This ignores the internal language composition of code-switched speech.

An utterance containing:

90% English + 10% Hindi

should not necessarily be evaluated against the same reference distribution as:

20% English + 80% Hindi

or:

50% English + 50% Hindi.

Utterance-Level Language Treatment

Many multilingual systems treat the complete utterance as belonging to one language.

However, code-switching can occur within a single sentence.

Therefore:

Utterance = English

or:

Utterance = Hindi

is an insufficient representation.

Existing Selective SER Research

Selective prediction and uncertainty-aware abstention already exist in Speech Emotion Recognition. Therefore, simply building an emotion model that “knows when it does not know” is not sufficiently novel.

The project source explicitly identifies this weakness: selective SER on conventional datasets has already been studied, and the research opportunity comes from moving the reliability problem into Hindi-English code-switched speech.

Research Gap

Research exists independently in:

- Speech Emotion Recognition;
- multilingual speech representation;
- code-switched speech processing;
- Out-of-Distribution detection;
- uncertainty calibration;
- selective prediction.

Therefore, SwitchSER cannot claim that any of these concepts is individually new.

The research gap investigated by SwitchSER is the joint problem of:

Sub-Utterance Language Identification

↓

Language-Aware Multilingual Emotion Representation

↓

Language-Mix-Conditioned OOD Detection

↓

Calibrated Uncertainty Estimation

↓

Selective Emotion Prediction

The specific research problem is that conventional OOD detectors calculate uncertainty without considering the linguistic composition of the input.

SwitchSER investigates whether an OOD score should depend on the utterance's language mixture.

For example:

80% Hindi + 20% English

should be evaluated primarily against the model's Hindi-related distribution.

Whereas:

20% Hindi + 80% English

should be evaluated primarily against its English-related distribution.

The central research question is:

Can language-mix-conditioned Out-of-Distribution detection improve the reliability of selective emotion recognition for Hindi-English code-switched speech compared with generic confidence and language-agnostic OOD methods?

A secondary research question is:

Can calibrated abstention reduce high-confidence emotion-classification errors while maintaining useful prediction coverage?

The proposed gap is specifically the conditioning of OOD detection on language composition rather than applying one global uncertainty score to every utterance.

Proposed Solution

SwitchSER is a selective Speech Emotion Recognition framework specifically designed for Hindi-English code-switched speech.

The complete workflow is:

Input Speech

↓

Frame-Level Language Identification

↓

Language Segmentation

↓

Language-Mix Ratio Estimation

↓

Language-Aware Multilingual Speech Encoder

↓

Emotion Representation

↓

Language-Conditioned OOD Detection

↓

Confidence Calibration

↓

Reliable Input?

If yes:

Predict Emotion

If no:

Abstain

The system first analyses the utterance at the frame or segment level.

For example:

Hindi: 65%
English: 35%

The speech is then processed using a multilingual speech encoder capable of representing both languages.

Instead of computing one generic OOD score, SwitchSER calculates language-aware uncertainty.

Conceptually:

$$\text{OODSwitch} = \alpha \times \text{OODHindi} + \beta \times \text{OODEnglish}$$

where:

- α = proportion or influence of Hindi;
- β = proportion or influence of English.

The final uncertainty score is calibrated.

If the input lies within a sufficiently reliable region:

Emotion prediction is returned.

If the input is unfamiliar or uncertain:

The system abstains.

The central design is therefore:

Classify when reliable. Abstain when unreliable.

Novel Contribution

The primary proposed contribution is a **language-mix-conditioned OOD detection mechanism for selective emotion recognition in code-switched speech**.

1. Sub-Utterance Language-Aware Representation

SwitchSER does not assign one language label to the complete utterance.

Instead, it identifies language information at the frame or segment level.

The representation therefore preserves the switching structure:

Hindi → Hindi → English → English → Hindi

rather than reducing the utterance to:

Mixed Language

2. Language-Conditioned OOD Scoring

A conventional detector calculates:

OOD(x)

SwitchSER investigates:

OOD(x | Language Composition)

The uncertainty estimate changes according to the Hindi-English mixture of the utterance.

3. Segment-Specific Distribution Comparison

Hindi-dominant segments and English-dominant segments are evaluated relative to their corresponding learned distributions.

The segment scores are then aggregated according to language composition.

4. Calibrated Selective Prediction

The system does not directly interpret raw neural confidence as reliability.

Temperature scaling is used to calibrate the final prediction confidence.

The final decision is:

Reliable and In-Distribution → Predict

Uncertain or Out-of-Distribution → Abstain

5. Coverage–Accuracy Optimization

The objective is not maximum raw classification accuracy at any cost.

Instead, the system optimizes the trade-off between:

Coverage: Percentage of inputs classified.

and:

Selective Risk: Error rate among inputs the model chooses to classify.

The proposed contribution is therefore:

**Frame-level language awareness → language-conditioned uncertainty →
calibrated OOD detection → selective abstention**

The source design identifies the language-conditioned OOD detector as the technical heart of the project.

Methodology

The primary experimental data should consist of Hindi-English code-switched emotional speech.

The original project design proposes:

- **IIITH-CSED** as the primary Hindi-English code-switched corpus;
- **IEMOCAP** for English monolingual comparison;
- **MELD** for held-out cross-domain evaluation.

The first stage performs language identification.

For an utterance divided into **T** frames:

$$\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T\}$$

each frame receives a language estimate:

$$l_t \in \{\text{Hindi, English, Other}\}$$

The system calculates a language-mixture vector:

$$\mathbf{m} = [\mathbf{rHindi}, \mathbf{rEnglish}]$$

where:

$$\mathbf{rHindi} + \mathbf{rEnglish} = 1$$

For example:

$$\mathbf{m} = [0.7, 0.3]$$

indicates approximately 70% Hindi and 30% English.

The speech is passed through a multilingual encoder such as XLS-R.

The encoder produces:

$$\mathbf{H} = \text{Encoder}(\mathbf{X}, \mathbf{L})$$

where:

- **X** = speech features;
- **L** = language-conditioning information;
- **H** = learned speech representation.

The original architecture proposes parameter-efficient adaptation using LoRA rather than unrestricted full-model fine-tuning.

The emotion classifier estimates:

$$\mathbf{P}(\mathbf{y} | \mathbf{H})$$

where **y** is the emotion class.

The OOD detector then calculates language-specific energy scores.

A standard energy score can be represented as:

$$E(\mathbf{x}) = -T \log \sum_i \exp(f_i(\mathbf{x})/T)$$

where:

- $f_i(\mathbf{x})$ = classifier logit;
- T = temperature.

SwitchSER extends this concept by calculating scores according to language composition.

Conceptually:

$$E_{\text{final}} = r_{\text{Hindi}} \times E_{\text{Hindi}} + r_{\text{English}} \times E_{\text{English}}$$

The final score is calibrated using temperature scaling.

The abstention policy is:

If $OODScore \leq \tau \rightarrow \text{Predict Emotion}$

If $OODScore > \tau \rightarrow \text{Abstain}$

where τ is selected using validation data.

The threshold is optimized according to a target coverage–accuracy trade-off rather than chosen arbitrarily.

Core Functionalities / Modules

Module 1: Code-Switched Speech Data and Language Analysis Pipeline

Creates the complete experimental corpus.

The module handles:

- dataset ingestion;
- audio standardization;
- silence removal;
- forced alignment;
- emotion-label harmonization;
- frame-level language identification;
- language-boundary detection;
- language-mix calculation;
- train-validation-test splitting.

The output for every utterance is:

Audio + Emotion Label + Frame-Level Language Tags + Language-Mix Ratio

The source architecture defines this module around IIITH-CSED, with language tagging, forced alignment and emotion-label harmonization.

Module 2: Language-Aware Multilingual Emotion Encoder

Uses a multilingual speech backbone capable of representing Hindi and English speech.

The module:

- loads the pretrained multilingual encoder;
- integrates language-conditioning signals;
- applies LoRA adapters;
- extracts multilingual acoustic representations;
- performs emotion classification.

The candidate backbone in the original design is XLS-R because it supports cross-lingual speech representation and can process both Hindi and English.

Module 3: Language-Conditioned OOD Detection and Calibration Engine

This is the primary research module.

It:

- separates language-aware representation statistics;
- calculates segment-level energy scores;
- conditions uncertainty on language composition;
- aggregates language-specific OOD evidence;
- applies temperature scaling;
- produces calibrated reliability scores.

Example output:

Emotion prediction: Angry
Hindi ratio: 72%
English ratio: 28%
OOD score: Low
Calibrated confidence: 87%

or:

Language mixture: Unfamiliar
OOD score: High
Decision: Abstain

Module 4: Selective Prediction and Abstention Engine

Determines whether the system should produce an emotion prediction.

It:

- receives calibrated OOD scores;
- applies the learned abstention threshold;
- calculates prediction coverage;
- enforces reliability requirements;
- returns either an emotion label or abstention.

The threshold is selected to optimize the coverage–accuracy trade-off rather than maximizing raw classification rate.

Module 5: Evaluation, Reliability Analysis and Research Dashboard

Evaluates the complete system.

It provides:

- emotion-classification metrics;
- OOD-detection metrics;
- calibration analysis;
- selective-risk curves;
- coverage–accuracy curves;
- language-mixture analysis;
- confusion matrices;
- abstention analysis;
- cross-domain evaluation;
- ablation studies.

The dashboard should allow researchers to inspect which language mixtures cause the highest error and abstention rates.

Evaluation and Baselines

The project should evaluate three separate capabilities:

1. emotion recognition;
2. OOD detection;
3. selective prediction.

The primary baselines should be:

Baseline 1: Monolingual SER

An English-oriented SER system that is forced to classify every utterance.

This measures how severely conventional models fail on code-switched speech.

Baseline 2: Multilingual SER Without Abstention

Uses the multilingual encoder but always predicts an emotion.

This isolates the effect of multilingual representation.

Baseline 3: Confidence Thresholding

Abstains when maximum softmax confidence falls below a threshold.

This tests whether generic confidence is sufficient.

Baseline 4: Language-Agnostic Energy-Based OOD Detection

Uses one global OOD score for the complete utterance.

This is the most important baseline because it isolates the contribution of language conditioning.

Baseline 5: SwitchSER Without Calibration

Uses language-aware OOD detection but does not apply temperature scaling.

This measures the value of confidence calibration.

Baseline 6: Full SwitchSER

Uses:

- frame-level language analysis;
- multilingual speech representation;
- language-mix-conditioned OOD detection;
- temperature calibration;
- selective abstention.

The original evaluation design compares forced prediction, naive thresholding, full SwitchSER and an oracle upper bound.

The primary metrics should include:

Emotion Recognition

- Accuracy;
- Macro F1-score;
- Unweighted Average Recall;
- class-wise F1-score.

OOD Detection

- AUROC;

- AUPRC;
- FPR@95TPR;
- OOD detection accuracy.

Calibration

- Expected Calibration Error;
- Brier Score;
- Negative Log-Likelihood.

Selective Prediction

- coverage;
- selective risk;
- risk–coverage curve;
- Area Under the Risk–Coverage Curve.

Language-Mixture Robustness

- performance by Hindi-English ratio;
- performance by number of language switches;
- abstention rate by language mixture;
- error rate near switching boundaries.

The most important ablation studies should remove:

- frame-level language conditioning;
- language-mix weighting;
- calibration;
- abstention;
- LoRA adaptation.

Expected Results / Impact

The expected result is a Speech Emotion Recognition framework that makes fewer high-confidence errors on unfamiliar Hindi-English code-switched speech than conventional forced-prediction systems.

The intended experimental pattern is:

Method	Emotion Accuracy	OOD Detection	Calibration	Abstention
Monolingual SER	Low	No	Poor	No

Multilingual SER	Higher	No	Moderate	No
Generic OOD SER	Higher	Moderate	Moderate	Yes
SwitchSER	Higher	Stronger	Better	Selective

The exact numerical results cannot be predicted before experimentation.

The target research conclusion is:

Conditioning OOD detection on the linguistic composition of code-switched speech improves the reliability of selective emotion recognition compared with language-agnostic uncertainty and confidence-thresholding methods.

The practical impact includes more reliable emotion analysis for:

- Indian customer-service systems;
- call-centre analytics;
- conversational AI;
- mental-health support tools;
- voice assistants;
- social robotics;
- multilingual human-computer interaction.

The project is particularly relevant to India because code-switching is a normal communication pattern rather than an edge case. The source document also identifies a critical feasibility dependency: access to the IIITH-CSED corpus should be confirmed before committing to the project.

Keywords

Speech Emotion Recognition, Code-Switched Speech, Hindi-English Speech, Selective Prediction, Out-of-Distribution Detection, Language-Aware OOD Detection, Uncertainty Estimation, Confidence Calibration, Multilingual Speech Processing, XLS-R, LoRA, Energy-Based OOD Detection, Abstention, Risk–Coverage Trade-off, Reliable AI

Title

Raphael: A Local-First Outcome-Aware Temporal Memory System for Personalized Long-Horizon Decision Support

Alternative research-paper title:

Outcome-Aware Temporal Memory for Personalized Long-Horizon AI Decision Support

Background

Modern AI assistants are effective at answering individual questions, generating content, and assisting with short-term tasks.

However, meaningful human decisions do not occur as isolated prompts.

A person's goals, constraints, assumptions, decisions, actions, and outcomes evolve over months or years.

For example:

January

Should I choose a technically ambitious project?

The AI recommends the ambitious option because the user appears to have sufficient time.

March

Project delayed.

The original assumption about available implementation time was wrong.

July

Should I choose another technically ambitious project before a short deadline?

A conventional AI assistant may retrieve the previous project conversation because it contains similar words.

However, useful decision support requires more than remembering that the conversation occurred.

The system should understand:

Past Situation

↓

Goal

↓

Available Options

↓

Evidence

↓

Assumptions

↓

Decision

↓

Action

↓

Actual Outcome

↓

Lesson

The critical information is not simply:

“The user previously selected Project X.”

The useful information is:

“The user selected Project X because implementation time was assumed to be sufficient. The project was delayed because integration complexity was underestimated. That failed assumption is relevant to the current decision.”

Current AI memory systems primarily retrieve information according to:

- recency;
- semantic similarity;
- conversation history;
- stored user facts.

These mechanisms are useful for remembering information but are insufficient for learning from the relationship between past decisions and their consequences.

Raphael investigates whether explicitly representing **decision–outcome history** can improve long-horizon AI decision support.

The core project formulation is a structured memory chain:

Situation → **Goal** → **Options** → **Evidence** → **Assumptions** → **Decision** → **Action** → **Outcome** → **Lesson**.

Problem

Current AI assistants have a fundamental long-term memory problem.

Suppose a user asks:

“Should I build a very difficult AI project in two months?”

A conventional memory system may retrieve previous conversations containing:

- AI projects;
- college deadlines;
- technical difficulty;
- research papers.

However, semantic similarity alone does not answer the most important question:

What happened when this user made a similar decision before?

Consider the historical episode:

Situation

Short project deadline.

Goal

Build a flagship research project.

Assumption

Core implementation can be completed in two weeks.

Decision

Choose the most technically complex option.

Outcome

Project delayed.

Cause

Integration and testing complexity were underestimated.

A conventional vector database may store all of this text.

But storing information is not equivalent to representing the relationship:

Assumption proved false

↓

Decision became risky

↓

Outcome was negative

↓

Lesson should influence future advice

The central problem is therefore:

How can an AI assistant maintain and retrieve structured relationships between a user's past situations, goals, assumptions, decisions, actions, and outcomes so that previous experience meaningfully influences future decision support?

The research problem is not simply:

Can an AI remember more?

It is:

Can an AI use the consequences of previous decisions to provide better future recommendations?

The source design explicitly defines the research question as whether structured decision–outcome history produces more temporally accurate, evidence-grounded and outcome-adaptive advice than conversation history or vector memory.

Existing Limitations

Short Context Windows

Basic AI assistants primarily use the current conversation.

Older information disappears when it falls outside the available context.

This creates:

Long-Term History → Lost

Conversation Summaries

Some systems summarize previous conversations.

For example:

“The user previously worked on several ambitious AI projects.”

This preserves general information but removes important causal details:

- Why was the project selected?
- What assumptions were made?
- What actually happened?
- Why did it succeed or fail?

Semantic Vector Memory

Modern memory systems commonly store text as embeddings.

When a new query arrives:

Current Query

↓

Embedding

↓

Similarity Search

↓

Retrieve Similar Memories

This is useful for answering:

What past information is textually similar?

It is weaker at answering:

Which previous decision had a structurally similar constraint?

or:

Which past assumption failed in a way relevant to this decision?

A past event may be highly relevant despite having low textual similarity.

Static User Profiles

AI systems may store facts such as:

User is a software engineer.
User prefers Python.
User wants to build startups.

These facts improve personalization but do not represent experiences.

A profile cannot express:

The user previously chose Option A because of Assumption B, but Outcome C showed that Assumption B was false.

Chronological Memory

A timeline can preserve:

Event A → Event B → Event C

However, temporal order alone does not represent causal or decision relationships.

The system must distinguish:

Occurred before

from:

Influenced

and:

Resulted in

No Outcome Feedback Loop

Most assistants provide advice and then move on.

The lifecycle is:

Question → Recommendation → End

There is usually no structured mechanism for:

Recommendation → Decision → Action → Outcome → Learning

Therefore, the assistant may repeatedly give advice based on assumptions that previous outcomes have already shown to be unreliable.

The core limitation is that existing memory systems may remember what the user said while failing to represent why a decision was made, what followed, and what should be learned from the result.

Research Gap

Research already exists in:

- conversational memory;
- vector databases;
- Retrieval-Augmented Generation;
- knowledge graphs;
- temporal knowledge graphs;
- personalized AI assistants;
- episodic memory for agents;
- long-term LLM memory.

Therefore, Raphael cannot claim novelty simply because:

“The AI remembers the user.”

That is not new.

The specific research gap investigated by Raphael is:

Outcome-aware temporal memory for long-horizon personal decision support.

Most memory systems optimize retrieval around:

Current Query ↔ Similar Past Information

Raphael investigates:

Current Decision

↕

Structurally Relevant Past Decision

↓

Original Assumptions

↓

Observed Outcome



Derived Lesson



Current Recommendation

The key distinction is:

Semantic Memory

What did the user say?

Episodic Memory

What happened?

Outcome-Aware Decision Memory

What was decided, why was it decided, what happened afterward, and what should that result change now?

The central research question is:

Does explicitly modelling relationships between situations, goals, assumptions, decisions, actions, and outcomes improve long-horizon AI decision support compared with prompt-only interaction and conventional vector-based memory?

Secondary research questions include:

Can outcome-aware retrieval identify relevant past experiences that semantic similarity retrieval misses?

Can previous failed assumptions be detected and surfaced when similar assumptions appear in new decisions?

Can an AI adapt future recommendations after observing the outcomes of previous advice?

The novelty must remain here. Claiming that Raphael “learns how the user thinks” would be vague, difficult to validate, and dangerously close to unsupported psychological profiling. The defensible claim is narrower: **Raphael learns from recorded relationships between decisions and their outcomes.**

Proposed Solution

Raphael is a local-first AI decision-support system with an outcome-aware temporal memory.

Instead of storing personal history only as messages or embeddings, Raphael represents important experiences as structured **decision episodes**.

Each episode contains:

Situation

↓

Goals

↓

Constraints

↓

Options

↓

Evidence

↓

Assumptions

↓

Decision

↓

Action

↓

Outcome

↓

Lesson

For example:

Situation

Short MCA project deadline.

Goal

Select a publishable flagship AI project.

Constraint

Limited GPU and implementation time.

Options

AuraGuard, FORGET-TS, SwitchSER.

Assumption

Complex infrastructure can be implemented within the deadline.

Decision

Choose AuraGuard.

Outcome

Implementation delayed.

Cause

Kubernetes testbed and telemetry integration required more time than estimated.

Lesson

Under short deadlines, infrastructure setup cost must receive higher weight.

Later, the user asks:

“Should I choose another infrastructure-heavy project?”

Raphael retrieves the past episode not merely because the words are similar but because the current decision contains a related structure:

Short Deadline + Infrastructure Complexity + Similar Assumption

The complete workflow is:

Current Decision

↓

Goal, Constraint and Option Extraction

↓

Outcome-Aware Temporal Memory Retrieval

↓

Relevant Past Decision Episodes

↓

Past Assumption and Outcome Analysis

↓

Current Option and Trade-Off Analysis

↓

Evidence-Grounded Recommendation

↓

User Makes Final Decision

↓

Outcome Later Observed

↓

Decision Episode Updated

↓

Lesson Extracted

↓

Future Decision Support Improved

Raphael remains human-in-the-loop.

The system provides:

- evidence;
- retrieved experiences;
- trade-offs;
- uncertainty;
- recommendations.

The user retains final decision authority. This full recommendation–decision–outcome–adaptation loop is the central architecture of the original proposal.

Novel Contribution

The primary proposed contribution is an **outcome-aware temporal memory architecture that retrieves and reasons over previous decision episodes using their assumptions and observed consequences.**

1. Structured Decision Episodes

Raphael does not store important experiences only as unstructured text.

Each significant decision is represented as:

$$D_i = \{S, G, C, O, E, A, D, X, R, L\}$$

where:

- **S** = Situation;
- **G** = Goal;
- **C** = Constraints;
- **O** = Options;
- **E** = Evidence;
- **A** = Assumptions;
- **D** = Decision;
- **X** = Action;
- **R** = Result or Outcome;
- **L** = Lesson.

This makes decision history explicitly queryable.

2. Hybrid Relevance Retrieval

Conventional vector retrieval primarily asks:

Which memory is semantically similar?

Raphael combines multiple relevance signals:

Relevance = Semantic Similarity + Temporal Relevance + Constraint Similarity + Assumption Similarity + Outcome Relevance

A past decision can therefore be retrieved even when its wording differs from the current problem.

3. Assumption–Outcome Tracking

This is the strongest component of the project.

Raphael explicitly records:

Assumption

I can complete the project in two weeks.

and later:

Outcome

Project required six weeks.

The system can mark:

Assumption Status: Contradicted by Outcome

Future recommendations can then surface this evidence.

4. Outcome-Aware Recommendation Adaptation

Most memory systems retrieve historical facts.

Raphael investigates whether outcomes should alter future recommendations.

The loop is:

Recommendation₁

↓

Decision₁

↓

Outcome₁

↓

Lesson₁

↓

Recommendation₂ is influenced by Outcome₁

5. Temporal Contradiction Handling

Personal state changes over time.

For example:

2025

User wants a traditional software job.

2026

User wants to build a startup.

Raphael should not treat both facts as equally current.

The system maintains:

- validity periods;
- superseded beliefs;
- temporal confidence;
- source timestamps.

6. Local-First Personal Memory

Sensitive long-term memory is stored locally under user control.

The local-first design is not the primary ML novelty, but it is a significant system contribution because the architecture may contain:

- personal goals;
- failures;
- financial constraints;
- relationships;
- career decisions;
- private documents.

The proposed contribution is therefore:

**Structured decision episodes + hybrid temporal retrieval +
assumption–outcome tracking + outcome-adaptive recommendation**

rather than simply:

“An AI assistant with memory.”

Methodology

The research begins by defining a structured decision-episode schema.

For every significant decision:

$D_i = (S_i, G_i, C_i, O_i, E_i, A_i, X_i, R_i, L_i, T_i)$

where T_i represents temporal metadata.

Decision Episode Extraction

Conversation or user input is analysed to extract:

- current situation;
- goal;
- constraints;
- available options;
- evidence;
- assumptions;
- uncertainty.

The extraction model converts unstructured conversation into structured memory.

Temporal Memory Graph

Decision episodes are connected using relationships such as:

SIMILAR_TO

DEPENDS_ON

ASSUMES

RESULTED_IN

CONTRADICTED_BY

SUPPORTED_BY

SUPERSEDES

LEARNED_FROM

For example:

Decision D_1

↓

ASSUMES

↓

“Two weeks is sufficient”

↓

CONTRADICTED_BY

↓

Outcome O_i

Hybrid Retrieval

For a new decision q , candidate historical episodes are ranked.

Conceptually:

$$\text{Score}(D_i, q) = \alpha S + \beta T + \gamma C + \delta A + \epsilon O$$

where:

- S = semantic similarity;
- T = temporal relevance;
- C = constraint similarity;
- A = assumption similarity;
- O = outcome relevance.

The weights can be tuned using validation experiments.

Outcome Capture

After a configurable time interval, Raphael requests or observes an outcome.

For example:

“You chose Project X three weeks ago. What happened?”

The system updates the episode:

Decision → **Action** → **Outcome**

Lesson Extraction

The system compares:

Expected Outcome

with:

Actual Outcome

and analyses:

- which assumptions were correct;
- which assumptions failed;
- which constraints were underestimated;

- which evidence proved reliable.

The generated lesson must remain linked to its supporting episode.

Recommendation Generation

For a new decision, the LLM receives:

- current situation;
- current constraints;
- relevant historical episodes;
- previous outcomes;
- failed assumptions;
- external evidence where available.

The output should contain:

- options;
- trade-offs;
- relevant historical evidence;
- uncertainty;
- recommendation.

The system then records the user's actual decision and continues the cycle.

Core Functionalities / Modules

Module 1: Personal Data Ingestion and Decision Episode Extraction Engine

Transforms unstructured personal information into structured decision episodes.

It handles:

- conversation ingestion;
- note ingestion;
- goal extraction;
- constraint extraction;
- option extraction;
- assumption identification;
- decision detection;
- evidence linking;
- confidence scoring.

Output:

Situation + Goal + Constraints + Options + Evidence + Assumptions + Decision

Module 2: Outcome-Aware Temporal Memory Graph

Stores personal history as an evolving temporal graph.

It manages:

- decision episodes;
- temporal relationships;
- causal links;
- assumption–outcome relationships;
- contradiction links;
- superseded information;
- provenance;
- timestamps.

The graph supports queries such as:

Which previous decisions involved short deadlines?

Which assumptions repeatedly failed?

Which decisions produced positive outcomes under similar constraints?

Module 3: Hybrid Experience Retrieval and Relevance Ranking Engine

This is the primary retrieval research module.

It combines:

- semantic vector similarity;
- temporal relevance;
- constraint similarity;
- assumption similarity;
- outcome relevance;
- graph connectivity.

The module compares conventional semantic retrieval with Raphael's outcome-aware retrieval.

Module 4: Decision Reasoning and Evidence-Grounded Recommendation Engine

This is the main AI reasoning module.

It:

- analyses current options;

- retrieves relevant experiences;
- identifies previous failed assumptions;
- detects contradictions;
- generates explicit trade-offs;
- estimates uncertainty;
- produces evidence-grounded recommendations.

The recommendation should explain:

What is recommended?

Why?

Which evidence supports it?

Which past outcomes are relevant?

What remains uncertain?

Module 5: Outcome Tracking, Adaptation and Research Evaluation Platform

Closes the feedback loop.

It:

- records final user decisions;
- schedules outcome follow-up;
- captures actual results;
- compares expected and actual outcomes;
- updates assumption validity;
- extracts lessons;
- measures future recommendation adaptation;
- provides an experiment dashboard.

The source proposal defines the same continuous feedback loop: retrieve historical episodes, analyse previous outcomes and failed assumptions, generate a recommendation, then incorporate subsequent outcomes into future decisions.

Evaluation and Baselines

Raphael should not be evaluated by asking:

“Does the chatbot feel personalized?”

That is subjective and academically weak.

The project requires controlled longitudinal decision scenarios with known histories.

Each benchmark scenario should contain:

- historical events;
- goals;
- decisions;
- assumptions;
- contradictions;
- outcomes;
- later decision queries.

The primary baselines should be:

Baseline 1: Prompt-Only LLM

Receives only the current decision.

No memory.

Baseline 2: Full Conversation History

Receives all previous conversations within the available context window.

Baseline 3: Conversation Summary Memory

Receives compressed summaries of historical conversations.

Baseline 4: Conventional Vector Memory

Retrieves the most semantically similar historical chunks.

This is the most important baseline.

Baseline 5: Temporal Memory Without Outcomes

Stores structured events and timestamps but does not explicitly model decision consequences.

Baseline 6: Raphael Without Assumption Tracking

Uses outcome-aware episodes but removes explicit assumption–outcome relationships.

Baseline 7: Full Raphael

Uses:

- structured decision episodes;
- temporal graph memory;

- hybrid retrieval;
- assumption–outcome tracking;
- contradiction handling;
- outcome-adaptive recommendation.

The source proposal already defines prompt-only and vector-memory systems as the primary experimental comparisons.

The primary evaluation metrics should include:

Memory Accuracy

- factual recall accuracy;
- temporal accuracy;
- contradiction resolution accuracy.

Retrieval Quality

- Precision@K;
- Recall@K;
- Mean Reciprocal Rank;
- relevant-experience retrieval rate.

Decision Support Quality

- evidence grounding;
- relevant historical evidence usage;
- trade-off completeness;
- unsupported claim rate.

Outcome Adaptation

- failed-assumption retrieval rate;
- recommendation-change rate after negative outcomes;
- repeated-error avoidance rate.

Longitudinal Consistency

- contradiction rate;
- obsolete-memory usage rate;
- temporal consistency score.

The critical experiment is:

Before Outcome

Model recommends Option A.

Then historical evidence records:

Option A failed because Assumption X was false.

Later, in a structurally similar decision:

Does the system retrieve that failure and adapt its recommendation?

If vector memory fails to retrieve the episode but Raphael succeeds, that is direct evidence for the proposed contribution.

Expected Results / Impact

The expected result is a personal AI decision-support system that uses previous decision outcomes more effectively than conventional semantic memory.

The intended experimental pattern is:

Method	Relevant Memory Retrieval	Outcome Adaptation	Temporal Accuracy
Prompt-Only LLM	Low	None	Low
Conversation History	Medium	Low	Medium
Vector Memory	High for Similar Text	Low–Medium	Medium
Temporal Memory	High	Medium	High
Raphael	Higher for Relevant Experience	High	High

The exact numerical results cannot be predicted before experimentation.

The target research conclusion is:

Explicitly modelling relationships between decisions, assumptions, and observed outcomes improves relevant-experience retrieval and long-horizon recommendation adaptation compared with memory systems based primarily on semantic similarity.

The practical impact includes:

- long-term personal AI assistants;
- career decision support;
- project planning;
- financial decision tracking;
- learning and goal management;

- founder and startup decision journals;
- personalized productivity systems.

The strongest version of Raphael is not:

“An AI that knows everything about you.”

That is a product pitch, not a research contribution.

The defensible version is:

An AI memory architecture that tests whether previous decision outcomes and failed assumptions can be explicitly represented and reused to improve future decision support.

This distinction matters. Raphael is a strong product concept, but its research-paper strength depends entirely on proving that **outcome-aware retrieval beats ordinary vector memory**. The underlying project definition already frames the evaluation around factual recall, temporal accuracy, relevant-experience retrieval, evidence grounding, contradiction handling and adaptation after outcomes.

Keywords

Long-Term AI Memory, Personalized AI, Decision Support Systems, Outcome-Aware Memory, Temporal Memory, Episodic Memory, Retrieval-Augmented Generation, Temporal Knowledge Graphs, Decision Episodes, Assumption Tracking, Outcome Adaptation, Hybrid Retrieval, Longitudinal AI, Local-First AI, Human-in-the-Loop AI, Evidence-Grounded Recommendation